

AI CSV Query System — Relatório Técnico & de Negócios

Desafio 4 (I2A2) — Plataforma de Autosserviço de BI e Consulta de CSVs via Agentes de IA

Data de Emissão: 28/07/2026

Status: Concluído (100% Pass)

Suíte de Testes: 34 Testes Aprovados

1. Proposta de Valor & Alinhamento de Negócio

O **AI CSV Query System** resolve o desafio de democratização de dados em ambientes corporativos. Ele capacita profissionais de negócios, auditores e gestores a realizar análises ad-hoc complexas diretamente em arquivos CSV usando linguagem natural, sem necessidade de aprender SQL ou depender de filas de chamados da equipe de BI.

Benefícios Chave de Negócio:

- Autosserviço de BI (Self-Service BI / No-Code):** Permite a exploração direta de dados operacionais sem necessidade de codificação ou consultas manuais.
- Redução do Tempo até o Insight (Time-to-Insight):** Reduz o ciclo de obtenção de respostas de dias/semanas para poucos segundos, disponibilizando resumos em texto, tabelas e gráficos interativos.
- Governança e Segurança de Dados (Zero Data Exposure):** Garantia de conformidade corporativa. Os arquivos CSV brutos permanecem isolados no DuckDB local e **nunca são enviados ou expostos ao LLM**. O modelo de IA recebe apenas os metadados de esquemas para síntese de consultas SQL.

2. Visão Geral da Arquitetura & Stack Tecnológico

A arquitetura foi projetada em 4 camadas desacopladas seguindo a premissa de que o LLM atua estritamente como orquestrador semântico, enquanto a consulta aos dados e a geração de visualizações ocorrem de forma 100% determinística via ferramentas isoladas.

Camada	Tecnologia	Função & Racional Arquitetural
Frontend	Streamlit	Interface reativa para upload de ZIP, chat interativo e exibição de esquemas e gráficos.
Backend	FastAPI	API RESTful assíncrona, com rotas /upload, /ask, /datasets e /health.
Orquestração	PydanticAI	Framework type-safe para agentes de IA com tool calling estruturado.
Motor OLAP	DuckDB	Banco SQL em memória de altíssimo desempenho para consultas em arquivos CSV.
Catálogo	SQLite	Persistência leve dos metadados e dicionários de dados dos datasets.
Visualização	Plotly	Geração de especificações JSON interativas para gráficos de barras, linhas e pizza.

3. Motor de Visualização & Geração de Gráficos

O componente `chart_tool` analisa semanticamente o resultado das consultas SQL e a intenção da pergunta do usuário para gerar especificações de gráficos declarativos em formato Plotly JSON:

Tipo de Gráfico	Propósito Analítico	Regra de Seleção Automática
Barras (bar)	Rankings de desempenho (Top N) e comparações entre categorias.	Perguntas de ranking (ex: 'Top 5 fornecedores') ou comparação categórica.
Linhas (line)	Séries temporais, evolução histórica e tendências ao longo do tempo.	Deteção de datas/meses ou termos como 'evolução', 'tendência' ou 'mensal'.
Pizza (pie)	Distribuição proporcional, participação percentual e composição.	Presença dos termos 'pizza', 'proporção', 'distribuição' ou poucas categorias (≤ 6).

Fluxo de Execução Desacoplado do Motor (Engine Flow):

[Etapa 1] Pergunta do Usuário (Linguagem Natural) Entrada livre em português (ex: 'Qual o total gasto em cada mês?').
[Etapa 2] PydanticAI Orchestrator Analisa a semântica e sintetiza consulta SQL Read-Only (Apenas metadados e esquemas enviados ao LLM).
[Etapa 3] DuckDB OLAP Engine (Processamento Local) Executa a consulta SQL em memória isolada local com altíssima performance (Zero Data Exposure ao LLM).
[Etapa 4] chart_tool (Plotly Engine) Inferência de tipos de colunas e intenção semântica para geração da especificação Plotly JSON (Barras, Linhas ou Pizza).

4. Segurança e Tratamento de Dados

- **Bloqueio de DDL/DML (SQL Injection):** A ferramenta `sql_tool` analisa a sintaxe SQL antes de executar. Comandos de manipulação ou destruição (ex: `DROP`, `DELETE`, `INSERT`, `ALTER`) são sumariamente rejeitados.
- **Prevenção contra Zip Slip:** A extração de pacotes ZIP valida o caminho de destino de cada arquivo (canonical path), garantindo que nenhum arquivo seja gravado fora do diretório temporário isolado.
- **Conversão de Tipos (VARCHAR → DOUBLE):** Tratamento automático de formatos numéricos brasileiros e cast explícito com `TRY_CAST` para prevenir erros de agregação (`SUM`) no DuckDB.

5. Validação das Perguntas de Demonstração (PRD)

Resultados obtidos com o dataset oficial `202401_NFs.zip` para as 5 perguntas de negócio do PRD:

#	Pergunta do PRD	Tipo Resposta	SQL Executado
1	Qual fornecedor recebeu o maior valor?	Texto + Valor	<code>SELECT fornecedor, SUM(TRY_CAST(valor AS DOUBLE)) AS total FROM nfs_202401 GROUP BY fornecedor ORDER BY total DESC LIMIT 1</code>
2	Qual produto apresentou o maior volume?	Texto + Valor	<code>SELECT produto, SUM(TRY_CAST(qtd AS DOUBLE)) AS total FROM nfs_202401 GROUP BY produto ORDER BY total DESC LIMIT 1</code>
3	Qual foi o total gasto em cada mês?	Gráfico Linha	<code>SELECT strftime('%Y-%m', data) AS mes, SUM(TRY_CAST(valor AS DOUBLE)) AS total FROM nfs_202401 GROUP BY mes ORDER BY mes</code>
4	Quais foram os 5 maiores fornecedores?	Tabela Top 5	<code>SELECT fornecedor, SUM(TRY_CAST(valor AS DOUBLE)) AS total FROM nfs_202401 GROUP BY fornecedor ORDER BY total DESC LIMIT 5</code>
5	Qual categoria teve maior crescimento?	Tabela / Gráfico	<code>SELECT categoria, SUM(TRY_CAST(valor AS DOUBLE)) AS total FROM nfs_202401 GROUP BY categoria ORDER BY total DESC LIMIT 5</code>

6. Cobertura da Suíte de Testes (Pytest)

Arquivo de Teste	Componente Testado	Qtd Testes	Status
<code>test_services.py</code>	Services (DuckDB, Catalog SQLite, ZipService)	8	PASSED
<code>test_upload.py</code>	API REST POST /upload & Exceções Customizadas	7	PASSED
<code>test_query.py</code>	PydanticAI Orchestrator & POST /ask	4	PASSED
<code>test_chart.py</code>	ChartTool (Plotly bar, line, pie)	5	PASSED
<code>test_frontend_module.py</code>	Cliente HTTP Streamlit (Upload, Chat, Datasets)	7	PASSED
<code>test_health.py</code>	Endpoint de Integridade GET /health	1	PASSED
<code>test_e2e.py</code>	Pipeline Integrado End-to-End	2	PASSED
TOTAL	Suíte Completa de Testes Integrados	34	100% PASSED

7. Instruções de Execução via Gerenciador uv

```
# 1. Instalar o gerenciador uv
curl -LsSf https://astral.sh/uv/install.sh | sh

# 2. Criar ambiente virtual e instalar dependências
uv venv && source .venv/bin/activate && uv pip install -r requirements.txt

# 3. Configurar variáveis de ambiente (.env)
cp .env.example .env # Inserir GOOGLE_API_KEY no .env

# 4. Executar Backend FastAPI (Terminal 1)
uv run uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload

# 5. Executar Frontend Streamlit (Terminal 2)
uv run streamlit run app_streamlit.py

# 6. Executar Suíte de Testes (Pytest)
```

```
uv run pytest -v
```

Relatório técnico e de negócios verificado para entrega do Desafio 4 (I2A2).